--> modality intent routing logic for query retrieval (show me, explain me or where in video)

Stage 1 — Video ingestion & pre-processing

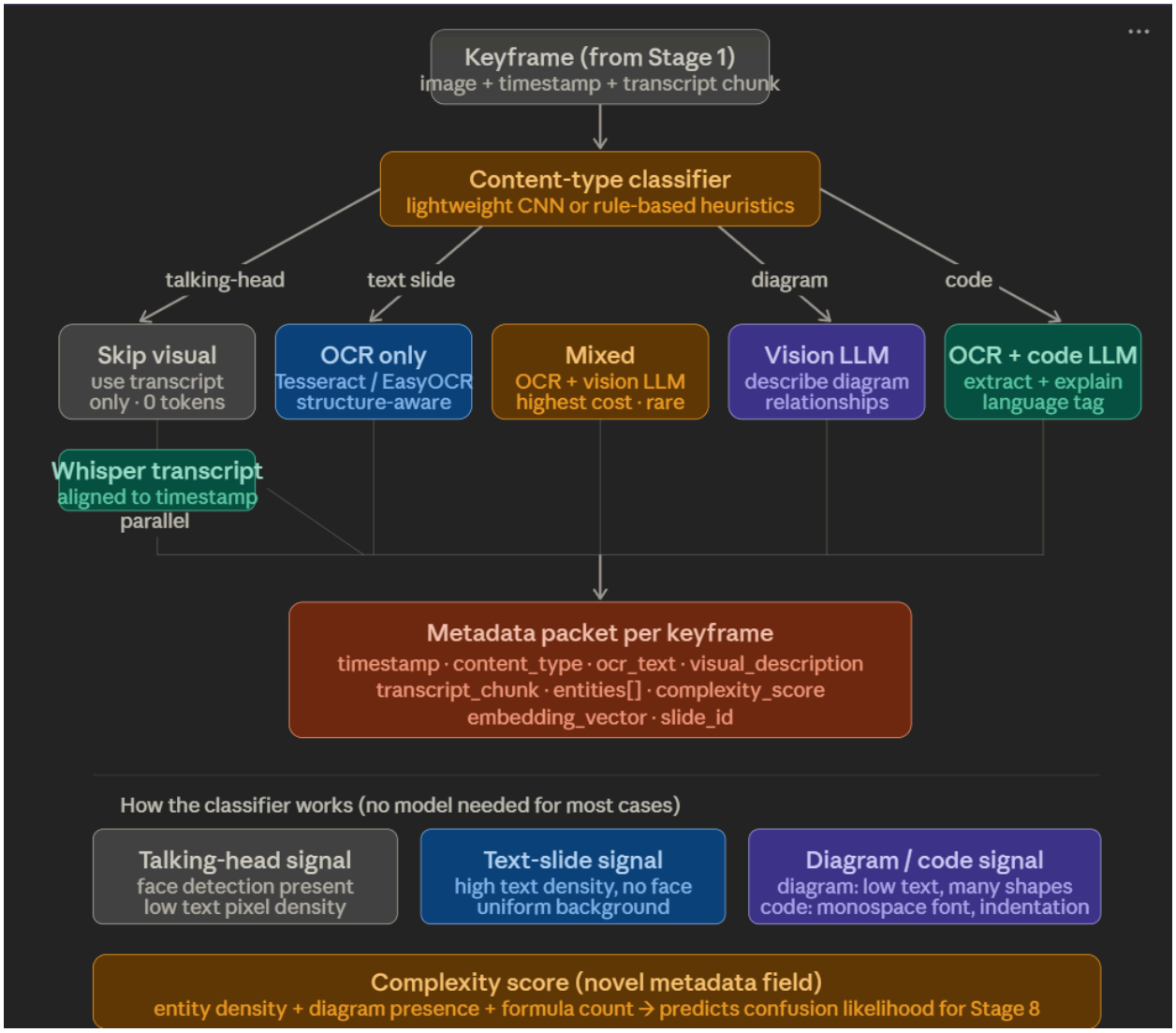
Frame extraction · audio separation · slide/topic change detection · scene boundary detection

Content-type classifiers (critical for cost reduction)(A frame showing a new code snippet needs OCR + LLM. A frame showing a hand-drawn diagram needs a vision model (e.g. GPT-4V or similar). A frame showing a dense equation needs vision + LaTeX-aware processing.)

SSIM for filtering frames

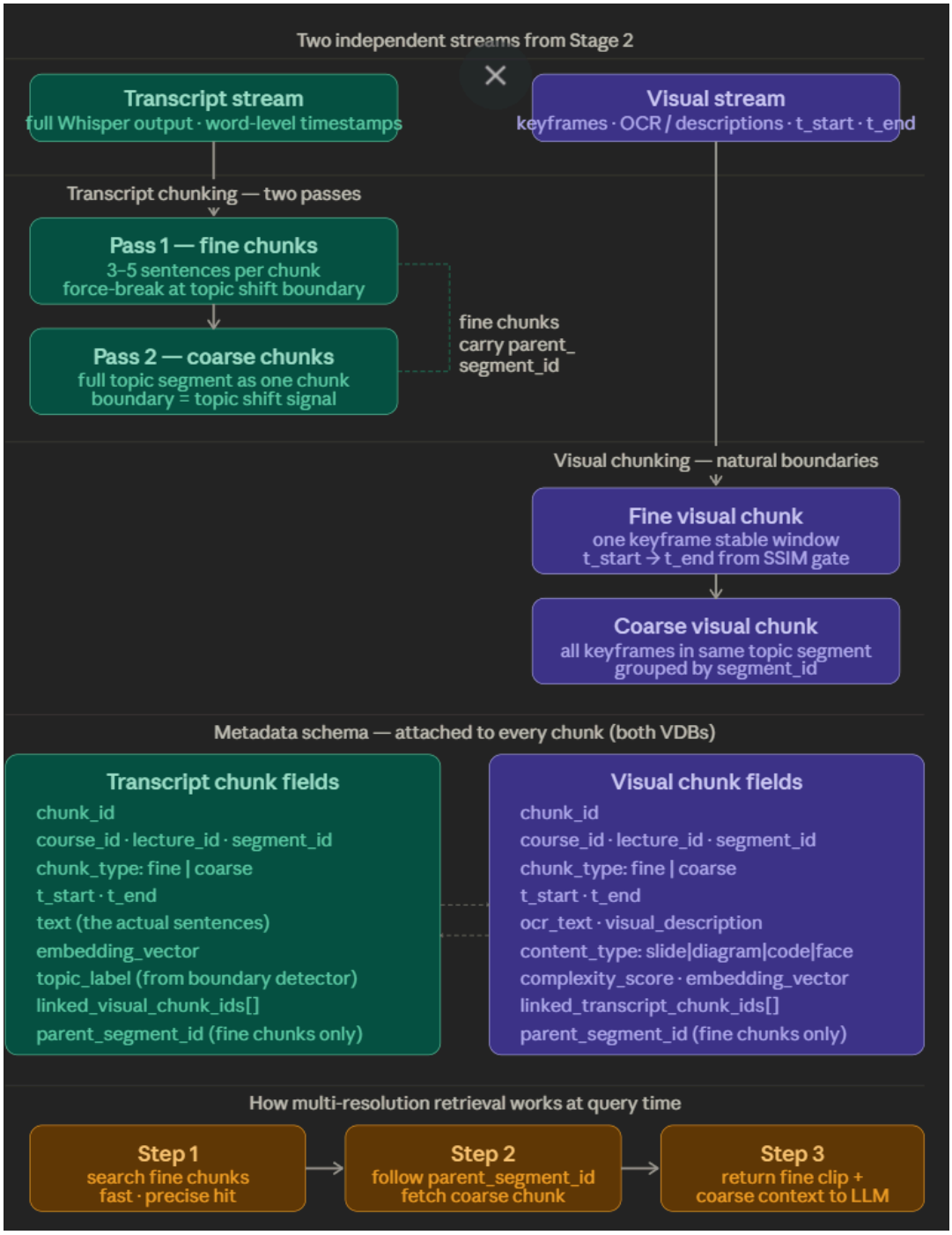
```
prev_frame = None
for frame in video_frames:
    if prev_frame is not None:
        score = ssim(frame, prev_frame)
        if score < THRESHOLD:
            save_keyframe(prev_frame, timestamp)
    prev_frame = frame

# flush at end
if prev_frame is not None:
    save_keyframe(prev_frame, end_timestamp)
```

Step 2 : Multimodal extraction

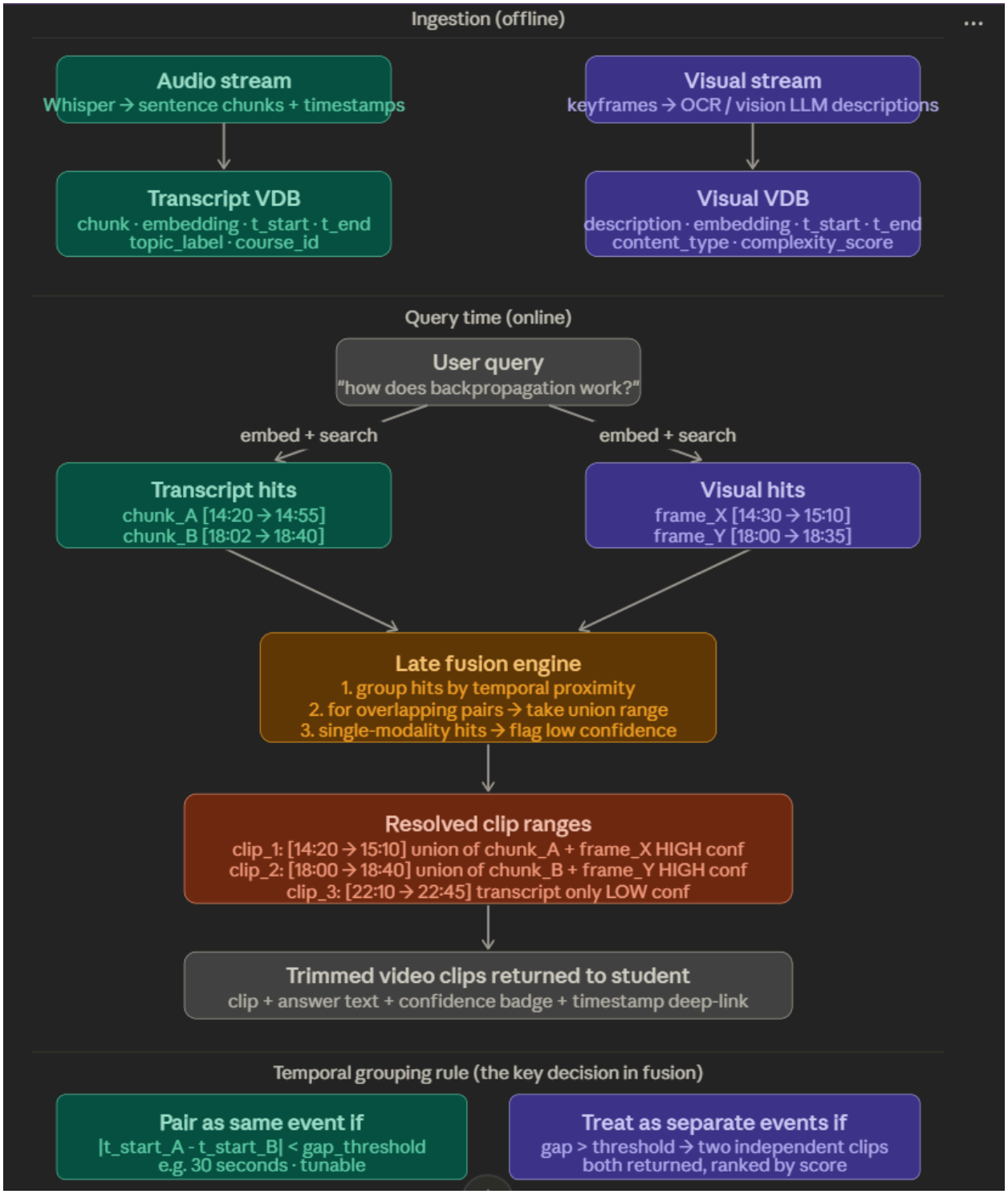


Metadata generation

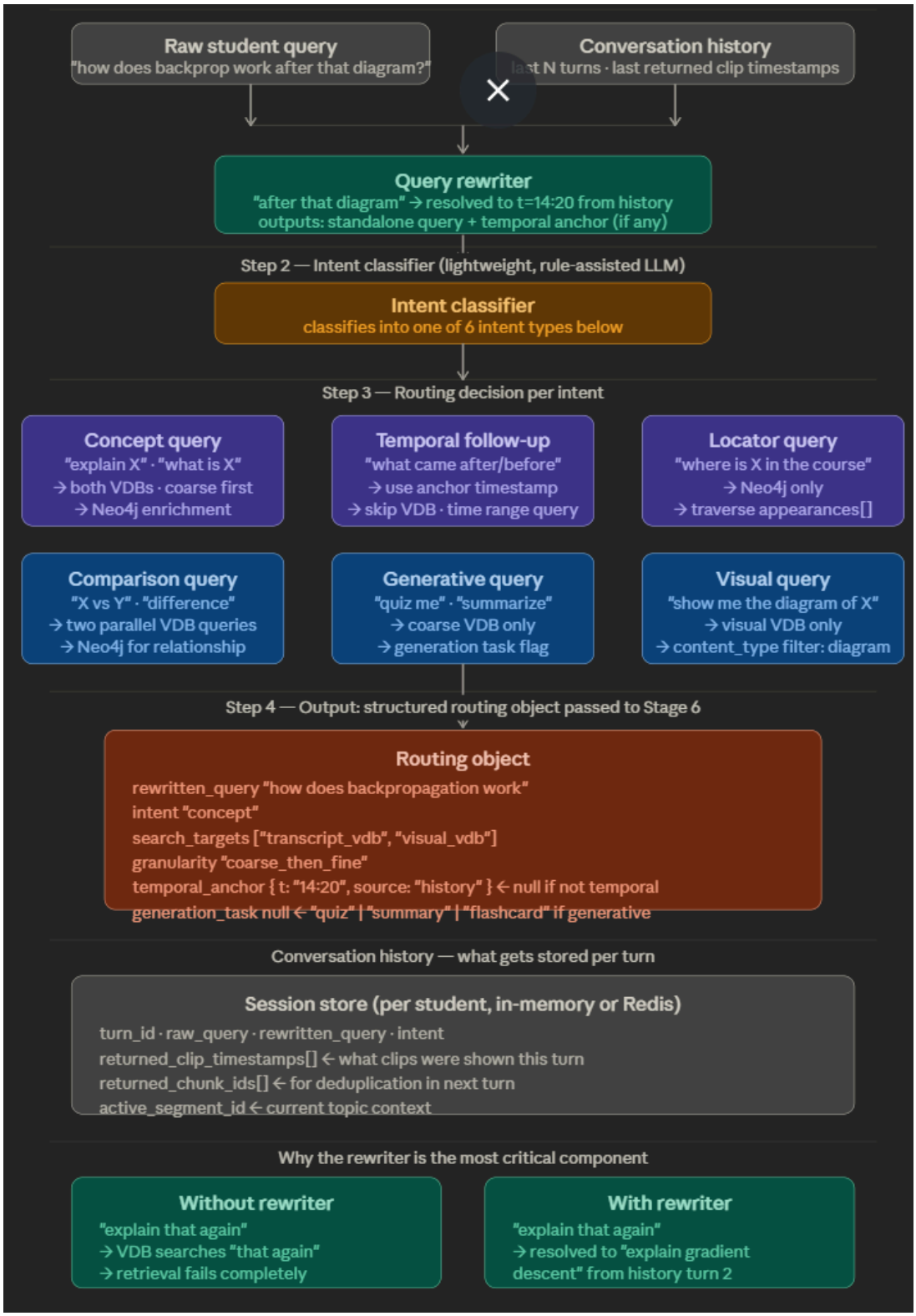
Stage 3 : Chunking and metadata attachment



Stage 4 — Indexing & storage, Multi-resolution vector indexing · hybrid graph+vector (Neo4j) · coarse→fine retrieval tags · confusion signal flags


Ingestion (offline)
Audio stream
Whisper → sentence chunks + timestamps
Visual stream
keyframes → OCR / vision LLM descriptions
Transcript VDB
chunk · embedding · t_start · t_end
topic_label · course_id
Visual VDB
description · embedding · t_start · t_end
content_type · complexity_score
Query time (online)
User query
"how does backpropagation work?"
embed + search
embed + search
Transcript hits
chunk_A [14:20 → 14:55]
chunk_B [18:02 → 18:40]
Visual hits
frame_X [14:30 → 15:10]
frame_Y [18:00 → 18:35]
Late fusion engine
1. group hits by temporal proximity
2. for overlapping pairs → take union range
3. single-modality hits → flag low confidence
Resolved clip ranges
clip_1: [14:20 → 15:10] union of chunk_A + frame_X HIGH conf
clip_2: [18:00 → 18:40] union of chunk_B + frame_Y HIGH conf
clip_3: [22:10 → 22:45] transcript only LOW conf
Trimmed video clips returned to student
clip + answer text + confidence badge + timestamp deep-link
Temporal grouping rule (the key decision in fusion)
Pair as same event if
|t_start_A - t_start_B| < gap_threshold
e.g. 30 seconds · tunable
Treat as separate events if
gap > threshold → two independent clips
both returned, ranked by score

Stage 5 — Query understanding Intent classification · temporal follow-up detection · modality routingStage



Intent → modality routing table (deterministic, no model needed)

Intent	Transcript VDB	Visual VDB	Neo4j
concept	YES · coarse→fine embed rewritten query	YES · coarse→fine same query vector	YES · after VDB enrich with neighbours
temporal follow-up	SKIP use anchor timestamp	SKIP use anchor timestamp	SKIP
locator	SKIP	SKIP	YES · only appearances[] lookup
comparison	YES · twice query A and query B	YES · twice query A and query B	YES · edge lookup A→B relationship
generative	YES · coarse only broad context needed	SKIP	SKIP
visual	SKIP	YES · fine only filter: diagram/code	SKIP

Embedding strategy — same query vector, different collections

One embedding call · two searches
embed rewritten_query once using a text embedding model (e.g. text-embedding-3-small)
reuse same vector for both transcript VDB and visual VDB searches — no second embed call

Why one text vector works for both VDBs

Transcript VDB
chunks are text → text embedding
query is text → same space · natural fit

Visual VDB
chunks are text descriptions of frames
query is text → same space · still works

Exception — visual queries with image input
if student uploads an image ("find slides like this") → use CLIP to embed query image
visual VDB must be indexed with CLIP embeddings to match · transcript VDB still uses text embed

for text queries: one embed model · one vector · two VDB searches run in parallel
parallel search cuts query latency roughly in half vs sequential

Query Rewriter for follow up questions on a time range

Prompt sent to rewriter LLM
HISTORY: turn1: "explain gradient descent" → clip [14:20–14:55]
turn2: "what is the learning rate in that?" → clip [14:22–14:40]
CURRENT: "explain that again but slower"

↓

Rewriter output — always structured JSON
rewritten_query: "explain learning rate"
temporal_anchor: { t_start: "14:22", t_end: "14:40", source: "turn2" }
is_followup: true
resolved_refs: { "that": "learning rate from turn2" }

↓

rewritten_query goes to VDB
"explain learning rate" → embedding → search

temporal_anchor bypasses VDB
direct time-range lookup on chunk store

Model choice for rewriter

GPT-4o-mini / Haiku
cheap · fast · good enough for rewriting

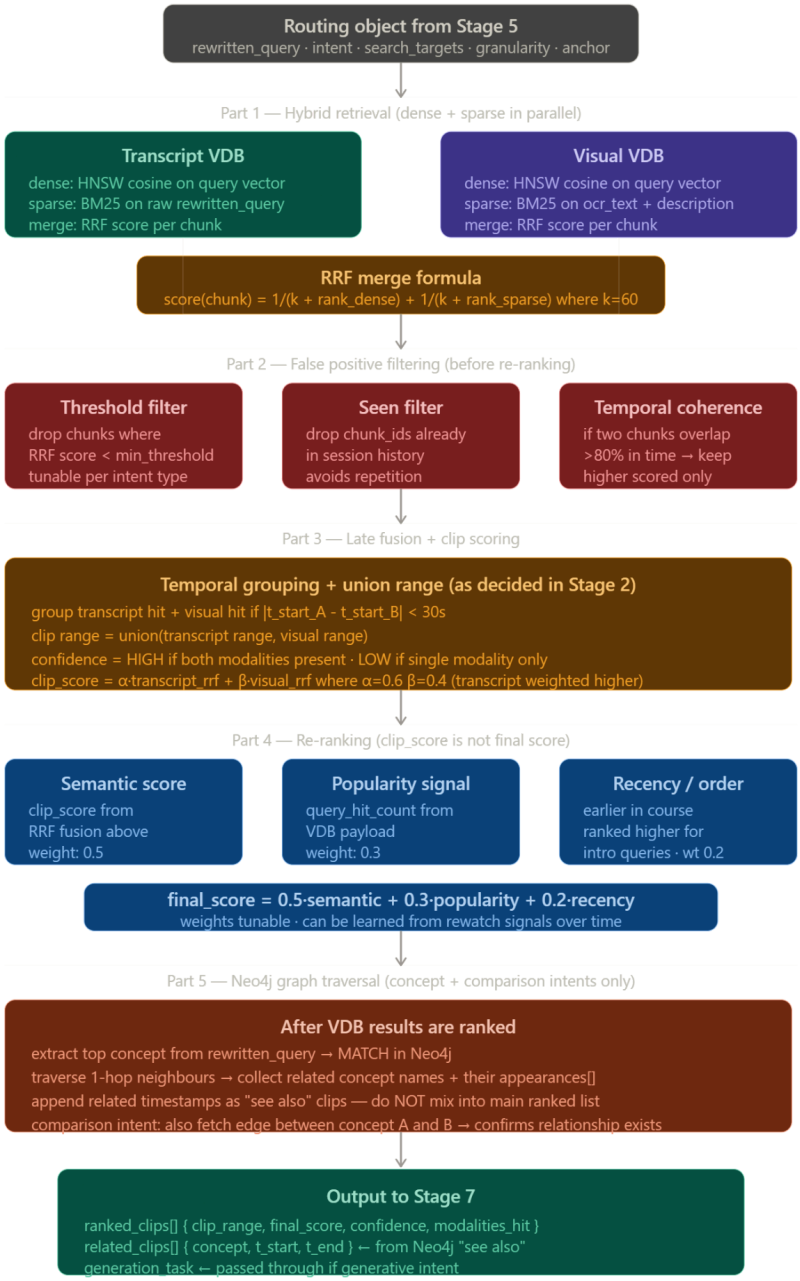
Max 5 history turns
beyond 5 → noise
truncate oldest first

Force JSON output
response_format:json
no parsing ambiguity

Skip rewriter entirely when
history is empty (first query in session) · no pronouns/references detected (cheap regex pre-check)

regex pre-check saves ~80% of rewriter LLM calls in practice

Stage 6 — Retrieval & re-rankingHybrid retrieval · false positive filtering · clip scoring · graph traversal for follow-upsStage



Stage 7 — Response generationAnswer + timestamped clip links · smart timestamps · multi-depth summaries · quiz generation

Stage 8 — Analytics & feedback loopConfusion detection · instructor dashboard · DB updates · recommendation signalsfeedback loop